

Sistemes Operatius

Tema 3: Concurrència, sincronització i exclusió mútua

© Xavi Canaleta, 2021

Tema 3. Concurrència, sincronització i exclusió mútua

1. Concurrència i compartició de dades
    1.1 Execució concurrent de processos
    1.2 Condicions de carrera
    1.3 Seccions crítiques
2. Exclusió mútua i sincronització
    2.1 El problema de l'exclusió mútua
    2.2 El problema dels productors i consumidors
    2.3 Conceptes associats: inanició (starvation), interbloqueig (deadlock) i equitat (fairness)
3. Eines d'Exclusió Mútua i Sincronització
    3.1 Semàfors
    3.1.1 Concepte i operacions
    3.1.2 Utilització de semàfors
    3.1.3 Patrons de sincronització i exclusió mútua
    3.1.4 Semàfors en C

1. Concurrència i compartició de dades

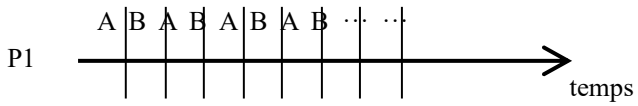
1.1 Execució concurrent de processos

Els problemes de **l'exclusió mútua i el deadlock** no són propis del món dels sistemes operatius ni tan sols de la informàtica. De fet podem trobar-ne força exemples en la vida quotidiana normal:

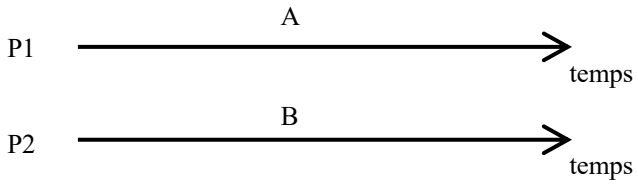
- Exemple d'una cruïlla on cada cotxe ha de cedir la dreta.
- Caixer automàtic (treure diners de dos llocs diferents contra el mateix número de compte quan hi ha 100 euros al compte corrent) .
- Dos threads que compten fins a 1 milió sobre una mateixa variable.

El problema de l'exclusió mútua i del deadlock apareix quan executem concurrentment o paral·lelament diversos processos i existeix comunicació entre aquests.

a) Concurrència: Dos o més processos competeixen per un mateix processador i existeix una compartició del recurs mitjançant la multiplexació en el temps. Per exemple tenim: 2 processos A i B; 1 processador P1



b) Paral·lelisme: Els processos s'executen en processadors diferents. Hi ha comunicació entre processos (compartir dades entre ells). Per exemple tenim: 2 processos A i B; 2 processadors P1 i P2



1.2 Condicions de carrera

Els problemes apareixen quan diversos processos volen compartir un recurs (memòria, perifèrics, ...) al mateix temps.

Exemple:

```
var  x:enter
fivar

procés A
   ...
   tmpA:=x          [1a]
   tmpA:=tmpA+1     [2a]
   ...
   x:=tmpA          [3a]
fiprocés
```

```
procés B
      ...
      tmpB = x        [1b]
      tmpB:=tmpB+1    [2b]
      ...
      x:=tmpB         [3b]
fiprocés
```

Aquests processos comparteixen la variable 'x'.
Si x=35 i executem concurrentment; pot passar:

	x = 35
[1a]	tmpA = 35
[2a]	tmpA = 36
[3a]	x = 36
[1b]	tmpB = 36
[2b]	tmpB = 37
[3b]	x = 37
	així estaria bé !

	x = 35
[1a]	tmpA = 35
[2a]	tmpA = 36
[1b]	tmpB = 35
[2b]	tmpB = 36
[3b]	x = 36
[3a]	x = 36
	malament !

1.3 Seccions crítiques

- No es pot accedir a variables compartides sense protegir l'accés.
- Les àrees que necessiten protecció s'anomenen: **Regions Crítiques**.

2. Exclusió mútua i sincronització

2.1 El problema de l'exclusió mútua

- El fet que quan un recurs està compartit, només pugui ser utilitzat per un procés al mateix temps, s'anomena: **Exclusió Mútua**.

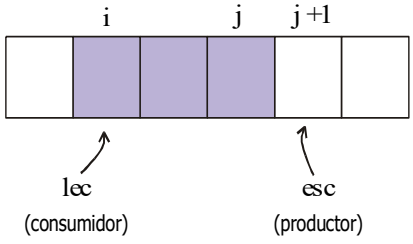
2.2 El problema dels productors i consumidors

Tenim un procés que produeix alguna cosa i un altre que consumeix. Hem de controlar quan volem produir i la cua està plena, o quan volem consumir i la cua esta buida. Per fer aquest tipus de problema disposem de:

- Tenim buffer compartit:

  consumidors → extreuen valors
  productors → guarden valors

- cua circular
- lec (1ª posició ocupada) → consumidor
- esc (1ª posició lliure) → productor
- compt. número d'elements que hi ha produïts però no els consumits



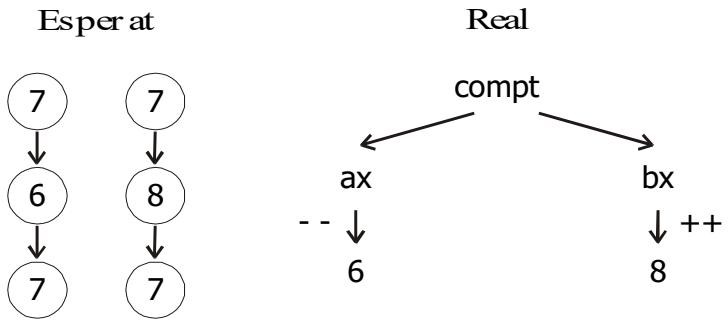
```
var
   buffer:array[0..MAX-1] de producte
   compt: enter
fivar

compt:=0
```

```
procés Productor()
var
  esc:enter
  C1:producte
fivar
   esc:=0
   mentre CERT fer
     C1:=produir()
     mentre compt=MAX//Bucle espera
     buffer[esc]:=C1
     esc:=(esc+1) mod MAX
     compt:=compt+1
   fimentre
fiprocés
```

```
procés Consumidor()
var
  lec:enter
  C2:producte
fivar
   lec:=0
   mentre CERT fer
     mentre compt=0 //Bucle espera
     C2:=buffer[lec]
     lec:=(lec+1) mod MAX
     compt:=compt-1
   fimentre
fiprocés
```

Aquest tipus d'exercici ens crea problemes. Uns exemples d'execució:



Per veure la causa dels problemes observem la situació en que ens trobem:

tenim unes variables compartides:

buffer, lec, esc } cap problema

compt. } perill !!!!

Al buffer no hi ha cap problema perquè llegeixen i escriuen en posicions diferents. Però amb la variable comptador si que pot haver-hi problemes; ja que és actualitzada per cada procés de manera independent, perdent el control d'accés. Per això hem de protegir l'accés al comptador, que es diu **REGIÓ CRÍTICA**.

A) Un problema a baix nivell pot no detectar-se a alt nivell.

Depenent de la traducció llenguatge alt nivell → a llenguatge màquina, es poden tenir problemes.

Exemple:

El bon funcionament del problema dependrà del compilador.

comptador ++ → inc cont
comptador -- → dec cont

podríem escriure com inc, aquesta si que és indivisible, però el compilador ens ha de permetre escriure-la. Aquest exemple només funciona amb la filosofia CISC:

80x86
MOTOROLA 68000, ...

B) Una solució que funciona correctament en un entorn concurrent, pot ser que no funcioni en un entorn paral·lel.

Exemple: Si tenim 2 CPU i fem un enable o disable, només activarà o desactivarà una CPU, per tant, existeixen problemes.

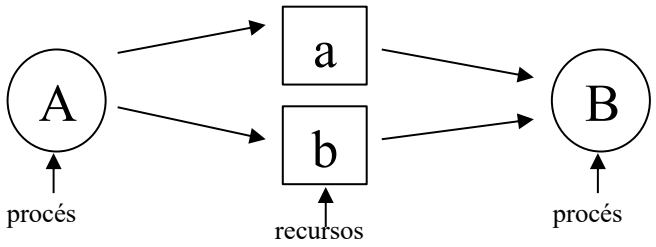
EI (enable)
DI (disable)

© Xavi Canaleta, 2021

2.3 Conceptes associats: inanició (starvation), interbloqueig (deadlock) i equitat (fairness)

Interbloqueig (deadlock)

Processos que no progressen i estan aturats sense despesa de temps de CPU.




```
procés A()
  DemanarRecurs1EnExclusió();
  DemanarRecurs2EnExclusió();
   /*Utilitza recursos */
   /* Reg. Crít. */
  AlliberaRecurs1();
  AlliberaRecurs2();
fiproces
```

```
procés B()
  DemanarRecurs2EnExclusió();
  DemanarRecurs1EnExclusió();
   /*Utilitza recursos */
   /* Reg. Crít. */
  AlliberaRecurs2();
  AlliberaRecurs1();
fiprocés
```

Possibles solucions:

1) no deixar que es produeixin
2) si es produeix, suprimir un procés

Inanició (starvation)

Un procés bloquejat a l'espera d'un recurs no s'executa mai per que hi ha altres processos que s'apoderen del recurs. No es compleix el 4t postulat de Dijkstra (Espera limitada).

Equitat (fairness)

Propietat global. Un sistema és equitatiu respecte un recurs quan tot procés que demana el recurs se li dóna en condicions semblats als altres.

3. Eines d'Exclusió Mútua i Sincronització

Les eines que ofereix el sistema operatiu permeten accedir a les regions crítiques i sincronitzar processos.

Aquestes eines del sistema operatiu eviten l'espera activa: bloquegen el procés que ha d'esperar i cedeixen la CPU a un altre procés preparat. En canvi, les solucions amb espera activa consumeixen temps de CPU sense fer treball útil.

Antecedents: esdeveniments sleep i wakeup

Els esdeveniments sleep i wakeup permeten evitar l'espera activa mitjançant el bloqueig i el desbloqueig de processos:

- **SLEEP: bloqueja el procés que l'executa sobre un esdeveniment indicat com a paràmetre. El procés roman bloquejat fins que es produeix aquest esdeveniment.**

- **WAKEUP: desperta els processos bloquejats per l'esdeveniment. Un procés s'executa mentre els altres romanen preparats o bloquejats segons el planificador i la semàntica de la primitiva.**

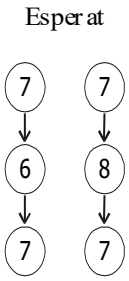
Exemple: Productors / Consumidors

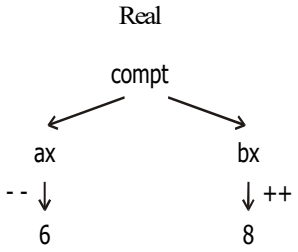
Idea:

- Quan el productor troba el buffer ple, es bloqueja fins que el consumidor n'extreu un element.
- Quan el consumidor troba el buffer buit, es bloqueja fins que el productor genera un element.

Utilitzem un vector per emmagatzemar els elements, un índex per a la següent posició d'escriptura i un altre per a la següent posició de lectura:

lec: primera posició ocupada; la utilitza el consumidor.
esc: primera posició lliure; la utilitza el productor.
Un comptador indica quants elements hi ha emmagatzemats.






```
void Productor() {

  for (;;) {
    C1 = produir();
    if (compt == MAX) sleep();
    buff[esc] = C1;
    esc = (esc + 1) % MAX;
    compt++;
    if (compt == 1) wakeup(Consumidor);
  }
}
```

```
void Consumidor() {

  for (;;) {
    if (compt == 0) sleep();
    C2 = buff[lec];
    lec = (lec + 1) % MAX;
    compt--;
    if (compt == MAX - 1) wakeup(Productor);
  }
}
```

Observacions:

- Aquesta solució no garanteix l'exclusió mútua sobre compt.
- Com que l'accés a compt no està protegit, es pot produir una condició de carrera.

Condició de carrera:
Situació en què dos o més processos accedeixen concurrentment a dades compartides, almenys un les modifica i el resultat final depèn de l'ordre d'execució.

3.1 Semàfors

3.1.1 Concepte i operacions

Les solucions al problema de la secció crítica presentades anteriorment no es poden generalitzar fàcilment a problemes més complexos. Per superar aquestes dificultats podem utilitzar una eina de sincronització anomenada Semàfors. Aquests són uns elements de sincronització i exclusió mútua introduïts per Dijkstra [1965].

Per exemple, si tenim 'n' processos que comparteixen un semàfor. Aquest semàfor *S* és una variable entera que, a partir de la seva inicialització, només pot ser accedida mitjançant 2 mètodes atòmics: ***wait*** i ***signal***.

Les funcions wait i signal s'executen de manera indivisible; és a dir, quan un procés modifica el valor del semàfor, cap altre procés pot modificar simultàniament aquest mateix valor.

Amb aquesta primera versió tenim un problema d'espera activa (*spin locks*). Que podem solucionar parametritzant wait i signal per que actuïn sobre qualsevol semàfor:

```
wait(S);      P(S);
signal(S);    V(S);

Passieren   to pass                Proberen  to test
Vrijmagen   to make free           Verhogen  to increment
```

Operacions wait i signal

```
tipus
   semàfor:  registre
                 compt: enter
                 cua: cua processos bloquejats
             firegistre
fitipus

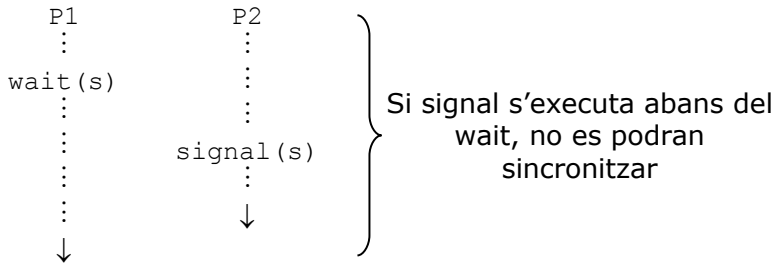
var
   s : semàfor
fivar

    proc wait(s:semafor)
       si s.compt > 0 llavors
         s.compt:=s.compt-1
       sino
         encuar(procés)
         bloquejar(procés)
       fisi
    fiproc
```

```
proc signal(s:semàfor)
   si ¬s.cua.Buida() llavors
     desencuar(&procés)
     desbloquejar(procés)
   sino
     s.compt:=s.compt+1
   fisi
```

Observacions:

1. El comptador sempre és positiu.
2. Necessitem un funció d'inicialització: init (S , n) on:
   - S → és el semàfor
   - n → núm. vegades que pots fer wait sense que es bloquegi
3. El valor al que inicialitzem el semàfor serà el número de processos que podran estar simultàniament en la regió crítica. Per exemple: init (S, 25)
4. Els semàfors també serveixen per sincronitzar processos.



Si volem fer exclusió mútua, hem d'inicialitzar com:

```
init (S,1);
```

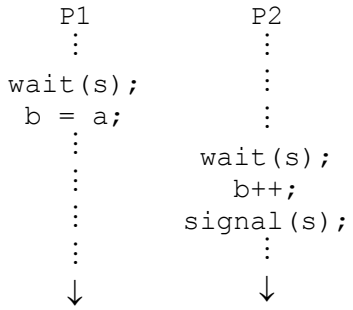
ara, si el que volem és utilitzar el semàfor de sincronització:

```
init (S,0);
```

5. El protocol establert per la regió crítica, és fer el següent ús:

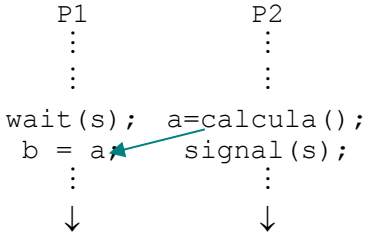
```
wait(s)
 /* Regió Crítica */
signal(s)
```

6. Vigilar amb l'ús. Hem de tenir compte amb l'ús que fem dels semàfors.



funciona correctament

però aquest exemple depèn de la inicialització



Funcionarà si fem init(S,0) però si inicialitzem el semàfor per utilitzar-lo per exclusió mútua: init(S,1), ens fallarà.

7. Hem solucionat el problema de l'exclusió mútua a nivell d'usuari; amb les comandes wait & signal del sistema operatiu.

3.1.2 Utilització de semàfors

L'ús del semàfor per entrar a la zona crítica per preservar l'exclusió mútua és el següent:

```
Semàfor S;          //Inicialitzem el semàfor
...
Wait(S);            //Activem el semàfor
.
. zona crítica
.
Signal(S);          //S'indica que es deixa la zona crítica
                    //  per que hi pugui accedir un altre procés
```

Aquesta filosofia dels semàfor funcionarà correctament sempre i quan tot els processos segueixin aquest ús del wait i el signal per entrar a la seva zona crítica.

3.1.3 Patrons de sincronització i exclusió mútua

Els semàfors també poden usar-se per solucionar diversos problemes de sincronització de processos.

```
P1:                     P2:
      ...                     ...
      c:= a+3                 z:=sqrt(c)
      ...                     ...
```

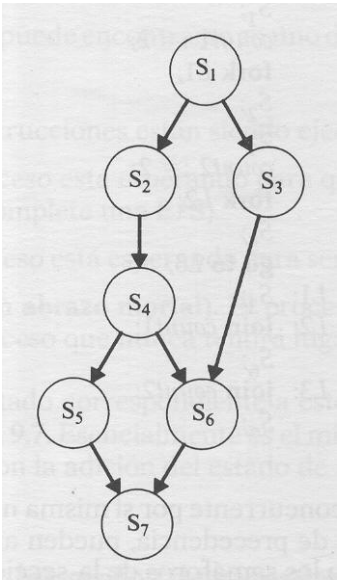
Cal sincronitzar perque P2 executi l'operació un cop acabada l'operació de P1.

```
S:Semàfor

Init(S,0)

P1:                     P2:
      ...                     ...
      c:= a+3                 wait(S)
      signal(S)               z:=sqrt(c)
      ...                     ...
```

Exercici pag. 341 Peterson Solucionat a pag. 367.




```
var
  a,b,c,d,e,f,g: semàfors     //inicialitzats a 0
fivar

procés P1              procés P2              procés P3
       S1                     wait(a)        wait(b)
       signal(a)              S2                    S3
       signal(b)              S4                    signal(e)
fiprocés                      signal(c)      fiprocés
                              signal(d)
                       fiprocés

procés P4              procés P5              procés P6
       wait(c)        wait(d)        wait(f)
       S5                     wait(e)        wait(g)
       signal(f)              S6                    S7
fiprocés                      signal(g)      fiprocés
                       fiprocés
```

Exemple: Productors / Consumidors

```
const
 MAX 50
ficonst
var
 buff: array[1..MAX] de element
 lec: enter                              /* posició lectura */
 esc:enter                               /* posició escriptura */
 mutex1, mutex2, buit, ple: semàfor
fivar

init (mutex1, 1);   /* exclusió mútua */
init (mutex2, 1);   /* exclusió mútua */
init (buit, MAX);  /* número de posicions lliures */
init (ple, 0);     /* número d'elements en el buffer */
lec = 0; esc = 0;
```

```
procés productor()
mentre CERT fer
  c1 := produir();
  wait(buit);
  wait(mutex1);
  buff[esc] := c1;
  esc := (esc+1) mod MAX;
  signal(mutex1);
  signal(ple);
fimentre
fiprocés
```

```
procés consumidor()
mentre CERT fer
  wait(ple);
  wait(mutex2);
  c2 := buff[lec];
  lec := (lec+1) mod MAX;
  signal(mutex2);
  signal(buit);
fimentre
fiprocés
```

mutex1: assegura que només hi ha un productor treballant sobre 'esc': `init(mutex1, 1);`

mutex2: assegura que només hi ha un consumidor treballant sobre 'lec': `init(sem2, 1);`

buit: el valor del semàfor indica el número de posicions lliures que hi ha al buffer. Si és 0 llavors el buffer està ple i el productor amb wait(buit) es bloqueja fins que el consumidor faci signal(buit).

ple: número de posicions ocupades en el buffer. Si és 0 llavors el buffer està buit i el consumidor amb wait (ple) es bloqueja fins que el productor faci signal(ple).

Semàfors binaris (o mutex)

El comptador només pot valer 0 o 1.

```
wait(s):
 si s.compt == 1 llavors
   s.compt = 0;
 sino
   encuar (procés)
   bloquejar (procés)
 fisi
```

```
signal(s):
 si <cua ¬ buida> llavors
   desencuar (procés)
   desbloquejar (procés)
 sino
   s.compt = 1;
 fisi
```

Per inicialitzar aquests semàfors: initb (S, n) on $n \in [0, 1]$

La diferència respecte els semàfors anteriors, és que els semàfor binaris NO memoritzen els signals. Els podem fer servir per garantir exclusió mútua o sincronització de processos.

3.1.4 Semàfors en C

```
var
   s: semàfor
   x: enter
fivar

Init(s,1)
```

```
Proces A()
var
 f:enter
fivar

Mentre CERT
   Llegeix(f)
   wait(s)
   x:=f
   signal(s)
   escriu(x)
fimentre
fiproces
```

```
Proces B()

 Mentre CERT
    wait(s)
    x:=x+1
    signal(s)
    escriu(x)
 fimentre

fiproces
```

```
//----------------PROCES A()------------------------------
//--------------------------------------------------------------------

//includes E/S
#include  <stdio.h>
//includes per memoria compartida
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
//includes per semafors
#include <sys/sem.h>

//Definicio de la union per semafors
union semun{
      int val; /* value for SETVAL */
      struct semid_ds *buf; /* buffer for IPC STAT, IPC SET */
```

```
      unsigned short *array; /* array for GETALL, SETALL */
};

//----------PROCES A()------------
int main(){
int f=0;
//mem compartida per x
int memid;
int *x;
// Declarem la union semun
union semun arg;
//Pels semafors
int semid;
unsigned short values[1];
struct sembuf operations[1];

  memid=shmget(0xCAFECAFE,sizeof(int),IPC_CREAT|IPC_EXCL|0600);
  if (memid>0){ /* Si la regio de memoria es nova */
      printf ("Regió compartida per x\n");
      /* Obtenim el punter a la memòria reservada */
      x=shmat(memid,0,0);
      /* Creem un nou conjunt de 2 semafors */
      semid=semget(0xCAFEFFFF, 1, IPC_CREAT|IPC_EXCL|0600);
      /* Definim l'array de valors per a inicialitzar */
      values[0]=1;
      arg.array = values;
      /* Actualitzem el valor de cada semafor */
      semctl (semid,0,SETALL,arg);
      while(f!=-1){
            printf("Valor f:");
            scanf("%d",&f);
            //La ida és jugar a comentar l'scanf anterior i
            //descomentar el de dins de la RC i veure com es
            //bloqueja el semafor.
            semctl (semid,0,GETALL,arg);
            printf("semafor abans de RC:%d\n",arg.array[0]);
            operations[0].sem_num=0; /* Treballem sobre el semafor 0 */
            operations[0].sem_op=-1; /* Volem decrementar 1 */
            operations[0].sem_flg=SEM_UNDO; /* Alliberar si CTRL+C */
            semop (semid, operations,1); /* Executem l'operació */
            /* regió compartida */
            //semctl(semid,0,GETALL,arg);
            //printf("Semafor a RC:%d\n",arg.array[0]);
            printf("Valor f2:");
            scanf("%d",&f);
            (*x)=f;
            printf("Valor actual de x a RC: %d\n",*x);
            /* regió compartida */
            operations[0].sem_num=0; /* Treballem sobre el semafor 0 */
            operations[0].sem_op=1; /* Volem incrementar 1 */
            operations[0].sem_flg=SEM_UNDO; /* Alliberar si CTRL+C */
            semop (semid, operations,1); /* Executem l'operació */
            semctl(semid,0,GETALL,arg);
            printf("Semafor despres RC:%d\n",arg.array[0]);
            printf("Valor actual de x: %d\n",*x);
        }
          /* Indiquem que el procés ja no necessita x*/
          shmdt(x);
          /* Indiquem que el procés ja no necessita la regió*/
          shmctl (memid, IPC_RMID, NULL );
```

```
            // Indiquem semafor ja no necessari
            semctl (semid,0,IPC_RMID,arg);
  }
  else  {   /* La regió de memòria ja exisitia */
        printf ("Regió array ja existent\n");
  }
  return 0;
}
```

```
//----------------PROCES B()------------------------------
//--------------------------------------------------------------------
#include  <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
#include <sys/sem.h>

//Definicio de la union per semafors
union semun{
       int val; /* value for SETVAL */
       struct semid_ds *buf; /* buffer for IPC STAT, IPC SET */
       unsigned short *array; /* array for GETALL, SETALL */
       /* Linux specific part: */
       //struct seminfo * buf; /* buffer for IPC INFO */
};

//----------PROCES B()------------
int main(){
int f=0;
//mem compartida per x
int memid;
int *x;
// Declarem la union semun
union semun arg;
//Pels semafors
int semid;
unsigned short values[1];
struct sembuf operations[1];

  memid=shmget(0xCAFECAFE,sizeof(int),IPC_CREAT|IPC_EXCL|0600);
  if (memid<=0){ /* Si la regio de memoria existent */
       printf ("Regió compartida per x del A\n");
       /* Obtenim el punter a la memòria reservada */
       memid=shmget(0xCAFECAFE,sizeof(int),0600);
       x=shmat(memid,0,0);
       /* Creem un nou conjunt de 2 semafors */
       semid=semget(0xCAFEFFFF,1,IPC_CREAT|IPC_EXCL|0600);
       if (semid<=0){ /* Si la semafor existeix */
          semid=semget(0xCAFEFFFF,1,0600);
          for(;;){
               semctl (semid,0, GETALL, arg);
               printf("Semafor abans RC:%d\n",arg.array[0]);
               operations[0].sem_num=0; /* Treballem sobre el semafor 0 */
               operations[0].sem_op=-1; /* Volem decrementar 1 */
               operations[0].sem_flg=SEM_UNDO; /* Alliberar si CTRL+C */
               semop (semid, operations,1); /* Executem l'operació */
               /* regió compartida */
               semctl (semid,0, GETALL, arg);
               printf("Semafor a RC:%d\n",arg.array[0]);
               (*x)=(*x)+1;
               printf("Valor actual de x a RC: %d\n",*x);
               /* regió compartida */
               operations[0].sem_num=0; /* Treballem sobre el semafor 0 */
               operations[0].sem_op=1; /* Volem incrementar 1 */
               operations[0].sem_flg=SEM_UNDO; /* Alliberar si CTRL+C */
               semop (semid, operations,1); /* Executem l'operació */
               semctl (semid,0, GETALL, arg);
               printf("Semafor despres RC:%d\n",arg.array[0]);
               sleep(3);
               printf("Valor actual de x: %d\n",*x);
           }
         }
         else {   /* Semafor no exisitia */
           printf ("semafor no existent\n");
           semctl (semid,0,IPC_RMID,arg);
         }
  }
  else  {   /* La regió de memòria no exisitia */
         printf ("Regió x no existent\n");
         /* Indiquem que el procés no necessita la regió*/
         shmctl (memid, IPC_RMID, NULL );
  }
  return 0;
}
```

Exercici: Problema dels lectors/escriptors [Curtois et al. 1971]

Un recurs de dades (com un registre d'una BBDD, un fitxer etc.) acostuma a haver-se de compartir entre diversos processos concurrents. Alguns d'aquests processos només fan consultes (lectures) del recurs, però n'hi ha d'altres que el modifiquen o actualitzen (escriptures).

Com és obvi, el recurs volem que preservi la integritat de dades. Per això, sembla lògic pensar que diversos processos lectors poden entrar a la vegada sense cap problema, però els processos escriptors (que modifiquen el contingut) han de tenir exclusivitat sobre el recurs a l'hora de realitzar les modificacions pertinents.

La variant que presentem aquí i que volem implementar és la més senzilla: cap lector caldrà que esperi si no és que un escriptor ha reservat ja el recurs. Això vol dir que cap lector s'espera pel recurs si n'hi ha d'altres llegint, independentment que un escriptor ho estigui fent.

Solució:

```
var
       Mutex,Escriure: semàfor
       QuantsLectors: enter
fivar

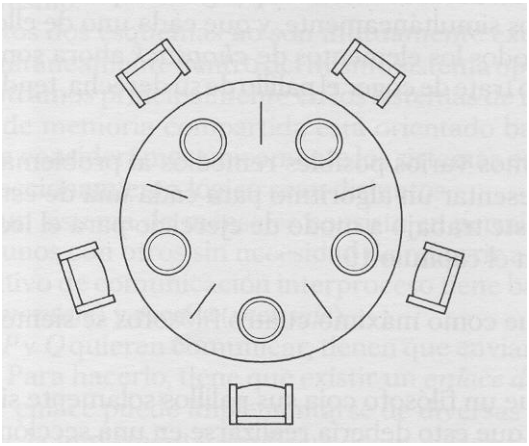
Init(Mutex,1)
Init(Escriure,1)
QuantsLectors:=0


procés Lector()
...
...
wait(Mutex)
       QuantsLectors:=QuantsLectors+1
       si QuantsLectors=1 llavors wait(escriure) fisi
signal(Mutex)
...
LlegeixRecursCompartit();
...
wait(mutex)
       QuantsLectors:=QuantsLectors-1
       si QuantsLectors=0 llavors signal(escriure) fisi
signal(mutex)
...
fiprocés

procés Escriptor()
...
...
wait(escriure)
       EscriuRecursCompartit()
signal(escriure)
...
fiprocés
```

Exercici: Problema del menjar dels filòsofs [Dijkstra 1965]

Cinc filòsofs xinesos dediquen les seves vides a pensar i menjar. Els filòsofs comparteixen una taula circular amb 5 cadires, cadascuna de les quals pertany a un d'ells. La disposició de la taula és la que mostra el dibuix següent:



Com podem veure hi ha 5 plats i 5 palillos. Quan un filòfof vol menjar ha d'agafar els dos palillos més propers a ell (esquerra i dreta) i posar-se a menjar. Com és evident, no podrà agafar un palillo que le seu veí estigui usant i caldrà que s'esperi.

Solució 1:

```
var
      Palillo: array[1..MAXPAL] de semàfor
fivar

per i:=1 fins MAXPAL
   Init(Palillo[i],1)
fiper


procés Filosof(i)

repeteix
      wait(palillo[i])
      wait(palillo[(i+1) mod MAXPAL])
      Menja();
      signal(palillo[(i+1) mod MAXPAL])
      signal(palillo[i])
fins que FALS
fiprocés
```

Pot provocar Deadlock!! Si tots agafen a la vegada el palillo de l'esquerra tots es queden esperant al de la dreta. Possibles solucions a implementar:

- Solucions asimètriques: els parells agafen esquerra – dreta i els senars agafen dreta – esquerra.
- Només agafar palillos si estan ambdós lliures (Secció Crítica).
- Permetre com a màxim 4 filòsofs asseguts a la taula alhora.

© Xavi Canaleta, 2021